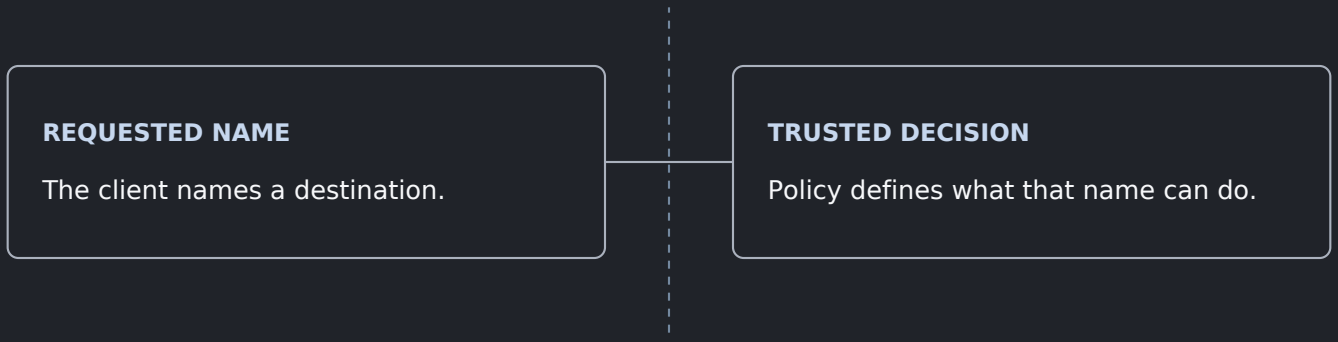
DESTINATION / PROVENANCE / EFFECT

HTTP Host Header Attacks Explained

Trace request authority from the edge
to the decision that trusts it.



Follow the name through proxies, caches and applications.
Prove the effect using real local HTTP, with fictional data.
Repair the decision and verify the legitimate path.

Haroon Shahid

First edition / Version 1.0 / 5 November 2026

Before you begin

HTTP Host Header Attacks Explained

Trace request authority from the edge to the decision that trusts it

By Haroon Shahid. First edition, version 1.0. 5 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under 05-ABOUT/Publication_License.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. The local lab uses real HTTP/1.1, TLS, HTTP/2 and fixed loopback services with fictional data and an in-memory mailbox. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Authority Yard Lab.

Contents

Before you begin 2

Contents 3

Introduction 4

Learning roadmap 5

01. One address, many websites 7

02. Read a URL without guessing 9

03. DNS and the network destination 11

Appendix A. Answer explanations 13

Sources and further reading 14

Introduction

Two requests arrive at Harbor Desk. Both connect to the same server and ask for the same path. The first request generates a password reset link at the portal. The second generates a link at a name the user never approved. Nothing about the token generator changed. The difference is which value the application trusted as its public address.

This book teaches how to find that decision. You will follow an ordinary URL through DNS, a connection, TLS, an HTTP parser, a reverse proxy, a cache and the application. Then you will change one input and check what actually happened. Some inputs select a site. Some are copied into a link. Some should have no effect at all.

The question that drives the book

Who supplied this value, what decision did it control, and how can I prove the effect? Ask that at each hop. It works whether you are looking at an HTTP/1.1 Host field, HTTP/2 :authority, an upstream certificate name or a cache key.

The case studies use Harbor Desk, a fictional support platform. Its lab, Authority Yard, runs on loopback with fixture users and reversible data. A password reset goes into an in-memory mailbox. A separate local listener proves whether a server made a request. A second tenant lets you test membership without touching anyone else's data.

What a professional result looks like

You can distinguish an altered string from a security failure. You can name the conditions that make that failure useful to an attacker. You can show the repair blocks the bad case while a legitimate request still works. The final capstone asks you to do all three without a list of payloads to copy.

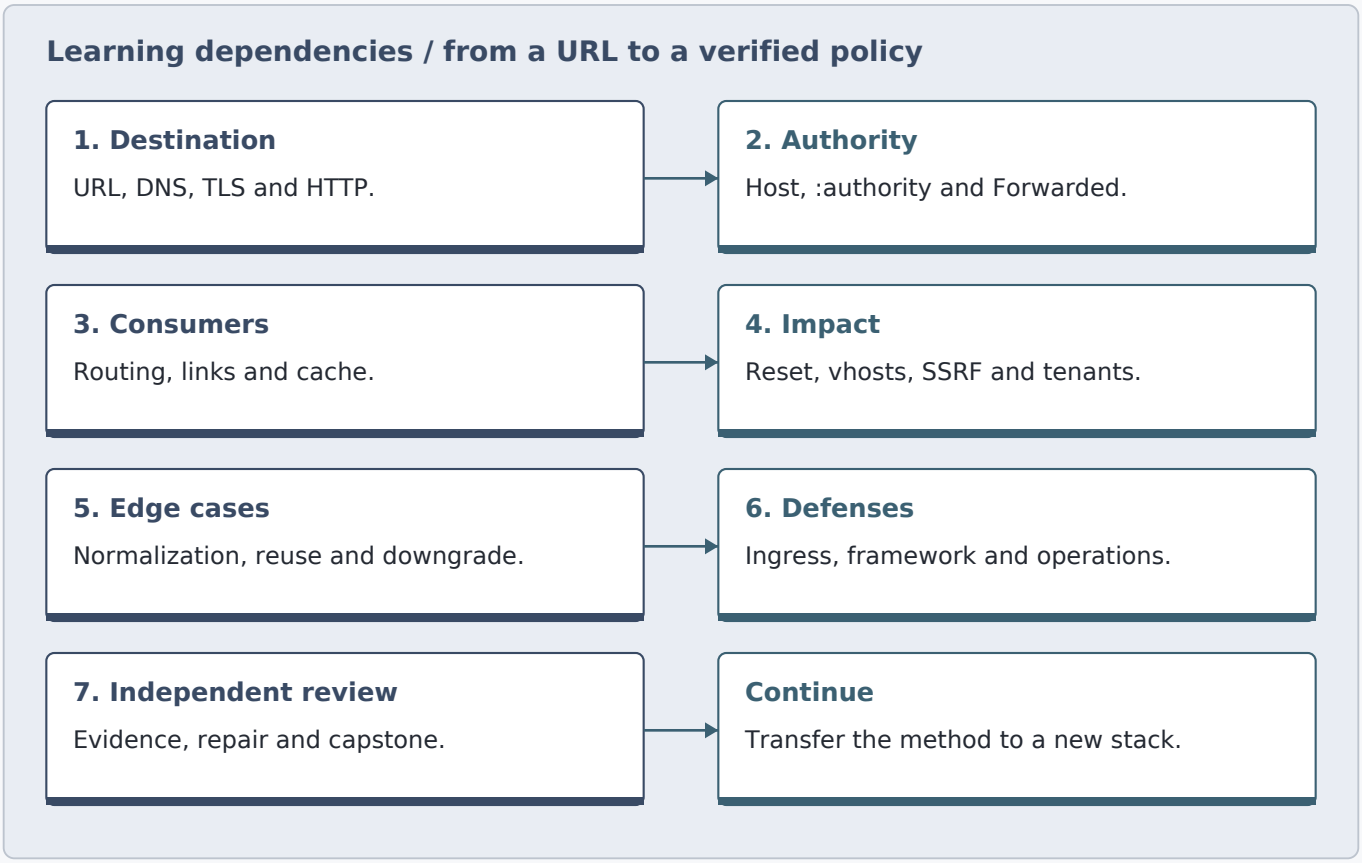
Learning roadmap

If you are...	Start here	Leave with...
New to HTTP	Chapters 1-12	A labeled request, a two-hop map and a trusted-source table.
Testing web apps	Chapters 13-30	Reproducible lab proofs, controls and bounded impact statements.
Building infrastructure	Chapters 19-24 and 31-36	Authority contracts, framework review questions and a regression plan.

Read the first part in order if ports, request targets and TLS are new. Predict the result before each practice exercise. Rebuild the diagram from memory afterwards. If a prediction was wrong, identify which layer you missed; do not merely copy the correct result.

Progress gate	Show you can...
Foundation	Explain why one address can serve two sites.
Protocol	Distinguish SNI, Host, :authority, forwarding and Origin.
Practitioner	Prove one altered identity link and one separate routing or cache effect.
Advanced	Reproduce a negative case and an allowed alternate across supported paths.
Transfer	Design a defensible contract for an unfamiliar edge without reusing lab assumptions.

The chapter openers state the objective. Each A figure teaches the mechanism; each B figure recaps input, interpretation, decision and effect. Use the separate workbook for missions, prediction prompts and worked cases.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

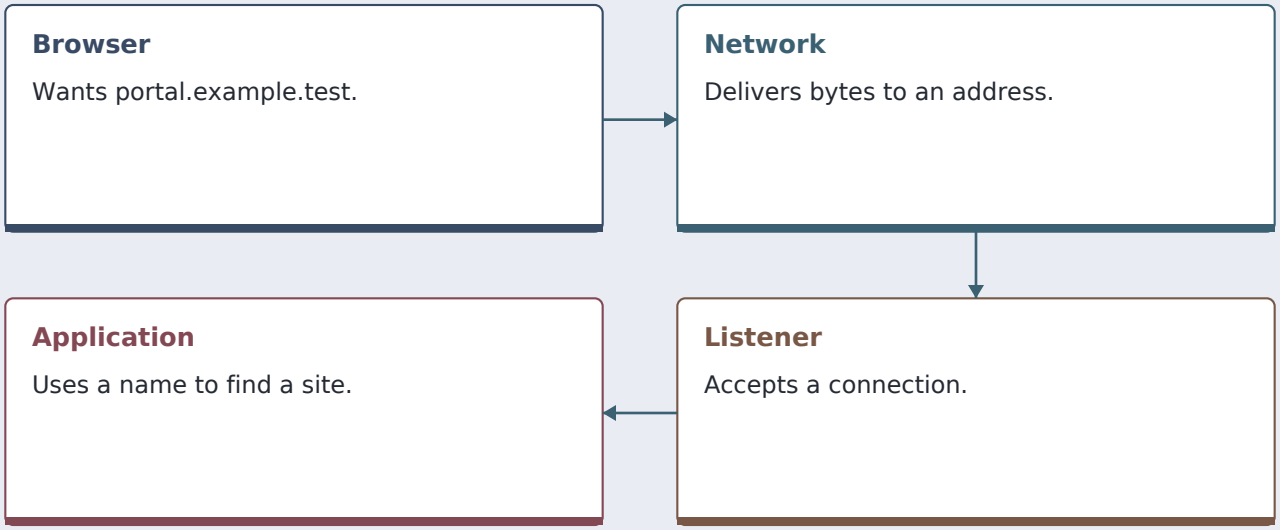
I. FROM A URL TO A REQUEST

01. One address, many websites

Learn to: Follow a website name from the address bar to the application.

Think of it this way: A delivery van needs the street address. The person inside still needs the name of the shop.

01A / Two decisions on one trip



An address delivers traffic to a listener. A name can help that listener choose the service.

From a name to a page

Enter `https://portal.example.test/help`. A URL tells the client how to ask for something. DNS, the domain name system, can help the client find an address for the name. An IP address identifies a network destination in a particular context. A port helps the operating system select a listener at that destination. In an ordinary HTTPS exchange, TLS protects the connection and the client checks the server identity before relying on it. Later, HTTP carries a request for a resource.

The application still needs to know which website you mean. Imagine Harbor Desk hosts both a customer portal and a public help center. Both can share an address and even share part of the same server program. The incoming name helps select a **virtual host**, a logical site that shares infrastructure.

What the systems know

The browser knows its URL. The network does not need to know that you want the help page. The edge receives the connection and reads the request. The application may see a rewritten request from that edge. One request can therefore have more than one useful name: the public name, a private upstream name, and a name associated with TLS. Writing them in separate columns will save time later.

Practice

Draw a client, one network address, and two sites behind it. Use one color for the connection and a label for the site name. Predict what stays fixed when a test client changes only the HTTP name. Compare with mission M01 when you reach the lab.

01B / Input, interpretation, decision, effect



Key takeaway. A network destination and an HTTP destination name answer different questions.

Check your understanding

1.1 Two names resolve to one address. Must they return the same page?	**1.2** Does sending a different Host move an open TCP connection?	**1.3** Why does a normal browser send a name after resolving it?

Answer explanations: Appendix A. Sources: H01, H02, H12

WORKED EXAMPLE / Change one destination layer

Imagine one listener answers portal and preview. On the same address and port, request the same path twice and change only Host. A route marker that changes from Pine to Vale demonstrates site selection. If the peer address also changed, repeat with a fixed dial address. You now know which variable your observation isolates.

02. Read a URL without guessing

Learn to: Identify a URL's scheme, hostname, port, path, query and fragment.

Think of it this way: A mailing label has separate fields for the city, building and room. Text in the notes is not the building address.

02A / A URL under a magnifying glass

`https://portal.example.test:8443/docs?tab=2#top`

Scheme	Host + port	Path + query	Fragment
https	portal.example.test :8443	/docs?tab=2	#top stays at client

Only the selected hostname and port enter the HTTP authority in this normal web URL.

Read each part

In `https://portal.example.test:8443/docs?tab=2#top`, the **scheme** is `https`, the **hostname** is `portal.example.test`, and the explicit **port** is 8443. The path is `/docs`; the query is `tab=2`. The fragment is `top`. For everyday HTTP URLs, this book calls the hostname plus optional port the **authority**. Generic URI authority can also include user information; a Host field is more restricted and must never be treated as an arbitrary URL.

Default ports matter. HTTPS uses 443 by default and HTTP uses 80. A comparison may normalize an explicit default port to the omitted form under a known scheme. An alternate port should only be allowed when it is explicitly part of the service policy. An IPv6 literal uses brackets in a web authority: `[::1]:8443`. Splitting a field at the first colon is not a general host parser.

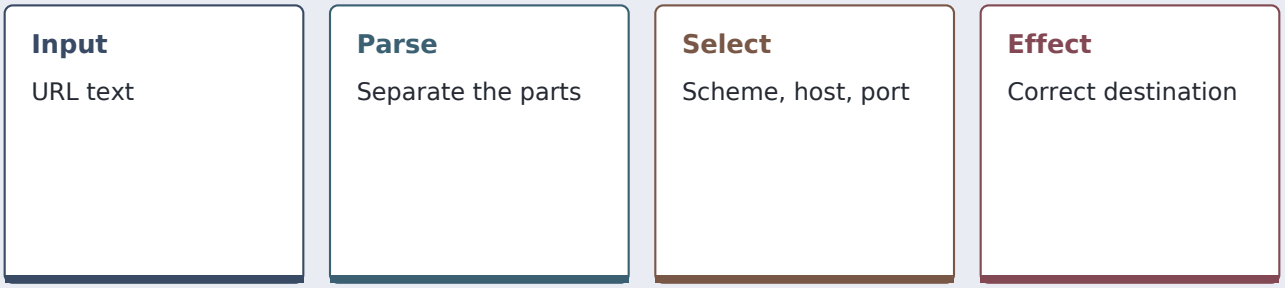
Where beginners lose the boundary

The name `portal.example.test` appears in `https://notes.invalid/?site=portal.example.test`. The browser connects to `notes.invalid`. The name can also appear before an at-sign as user information in a URL. Reject user information where you expect a host only. A parser describes what the input means. An allowlist decides whether you meant to permit it. Keep those jobs separate.

Practice

Underline the actual host in `https://portal.example.test@notes.invalid/a` and in `https://notes.invalid/portal.example.test`. Then write the path for each. Explain why neither example can be approved by looking for a trusted word.

02B / Input, interpretation, decision, effect



Key takeaway. Compare parsed authority to policy; a word appearing somewhere in a URL proves very little.

Check your understanding

2.1 What is the host in https://portal.example.test.note.invalid/help ?	**2.2** Does a normal HTTP request send #top from the URL as part of the target?	**2.3** Are https://portal.example.test and https://portal.example.test:8443 the same origin?

Answer explanations: Appendix A. Sources: H01, H07, H08

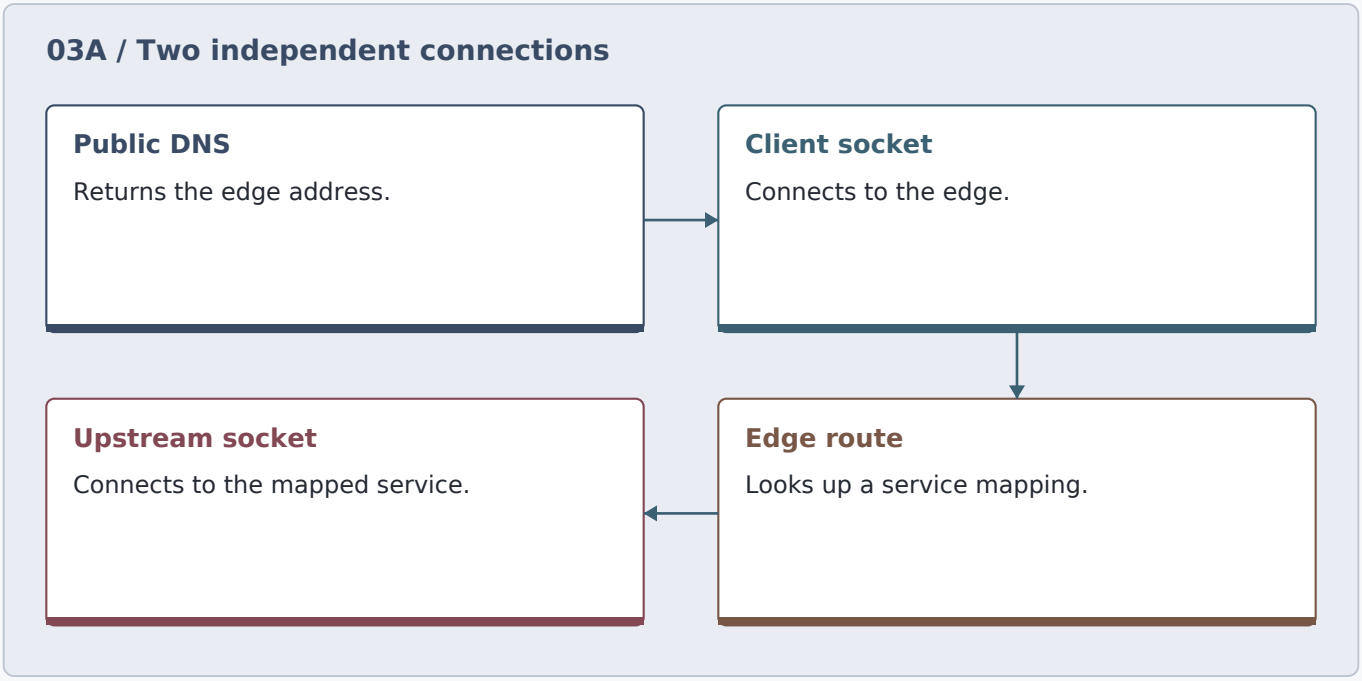
WORKED EXAMPLE / Calculate before comparing

Compare https://portal.example.test:443/docs and https://portal.example.test:8443/docs. Both have the same hostname; only the first uses the default HTTPS port. If an allowlist validates hostname alone but the result later determines a dial target, you have approved more destinations than you meant to.

03. DNS and the network destination

Learn to: Separate name resolution, selected address and the upstream target.

Think of it this way: A directory gives you the building address. It does not decide which doors you may open.



An edge normally makes its own upstream connection and can use a different name and address.

What a name can and cannot prove

DNS maps names to data, such as addresses. A CNAME can alias a name to another name. Multiple names can converge on one address, and one name can have multiple addresses. Proxies and operating systems also cache answers. A reverse proxy often makes a new connection after it has already accepted the client's connection. This second connection can use a separate resolver, network, service identity, and DNS search rules.

A raw HTTP client can connect to `127.0.0.1` and write `Host: portal.example.test`. No DNS record for that teaching name is necessary because the client has an explicit dial address. The target process still gets to decide what the header means. This is a useful testing setup: keep the destination constant, then change one authority input.

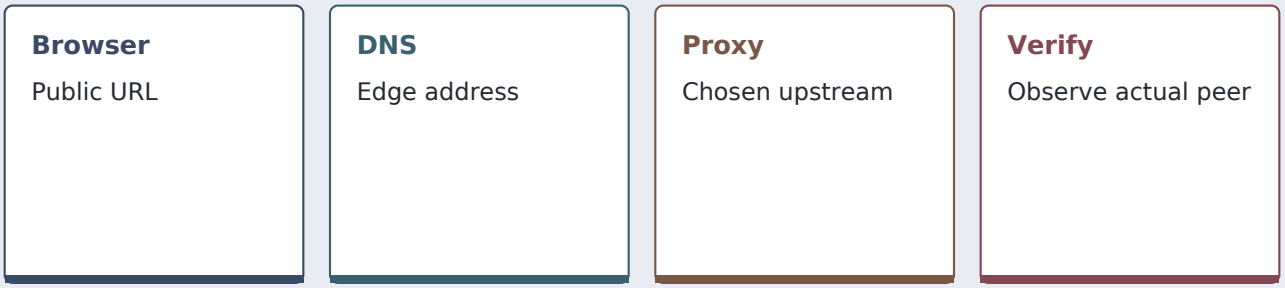
Why this matters for impact

If a suspected route returns your marker in a JSON error, the marker might have come from request logging. If an approved listener observes DNS, the service could have resolved your name without reaching it. A request logged by a controlled HTTP sink is stronger evidence: it shows a server-side HTTP action under those conditions. Its method, headers and source path still need separate verification before you make stronger claims.

Practice

Write three observations for a fixed lab input: a reflected string, a DNS query, and a matched HTTP listener log. Rank them by what each proves. For each, write one claim you still cannot make.

03B / Input, interpretation, decision, effect



Key takeaway. A reflected hostname is not evidence that a server resolved it or connected to it.

Check your understanding

3.1 Why can a nonexistent public name select a site in a local lab?	**3.2** The proxy logs that it resolved an input name. Did it send HTTP to it?	**3.3** Can multiple DNS answers change a proxy safety check?

Answer explanations: Appendix A. Sources: H01, H09, H16, H44

WORKED EXAMPLE / Name lookup versus connection

In one review the edge logs a lookup for the marker but a firewall denies every connection attempt. In another it connects to the fixed observer and sends a GET. Keep the first as DNS evidence, and tie the second to the HTTP journal. A third observation, the observer's body reaching the caller, is a different capability.

Appendix A. Answer explanations

Chapter 01: One address, many websites

1.1 Two names resolve to one address. Must they return the same page? No. A listener can serve many virtual hosts and choose a response using the request authority.

1.2 Does sending a different Host move an open TCP connection? No. It sends different bytes over the existing connection. A proxy may choose another upstream because of those bytes.

1.3 Why does a normal browser send a name after resolving it? The receiver may need the name to distinguish websites that share infrastructure.

Chapter 02: Read a URL without guessing

2.1 What is the host in https://portal.example.test.note.invalid/help? portal.example.test.note.invalid. The allowed name only occurs inside a longer hostname.

2.2 Does a normal HTTP request send #top from the URL as part of the target? No. The client handles the fragment separately.

2.3 Are https://portal.example.test and https://portal.example.test:8443 the same origin? No. Their effective ports differ.

Chapter 03: DNS and the network destination

3.1 Why can a nonexistent public name select a site in a local lab? A raw client can connect directly to a known loopback address and write that name in HTTP. No DNS lookup for the HTTP field is necessary.

3.2 The proxy logs that it resolved an input name. Did it send HTTP to it? Not necessarily. DNS resolution is distinct from TCP and from an HTTP request.

3.3 Can multiple DNS answers change a proxy safety check? Yes, if the proxy validates one answer but connects using another lookup or address. Bind policy to the actual destination.

Sources and further reading

Research cutoff: 5 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

H01 - HTTP Semantics, RFC 9110

Consulted. Section 7 establishes the target URI, Host and :authority, routing and misdirected requests.

https://www.rfc-editor.org/rfc/rfc9110.html#section-7

H02 - HTTP/1.1, RFC 9112

Consulted. Request-target forms, Host handling and message parsing. Distinguish malformed input from authority policy.

https://www.rfc-editor.org/rfc/rfc9112.html#section-3.2

H07 - WHATWG URL Standard

Consulted. Browser URL parsing, host forms, origins and normalization.

https://url.spec.whatwg.org/

H08 - URI Generic Syntax, RFC 3986

Consulted. Components of a URI, generic authority, literals, normalization and comparison.

https://www.rfc-editor.org/rfc/rfc3986.html#section-3

H09 - HTTP Caching, RFC 9111

Consulted. Storage, cache keys, matching, freshness and validation. Compare with actual intermediary settings.

https://www.rfc-editor.org/rfc/rfc9111.html

H12 - NGINX server names

Consulted. Name matching and selection at different request stages.

https://nginx.org/en/docs/http/server_names.html

H16 - OWASP WSTG: Testing for Host Header Injection

Consulted. Testing scope and attack categories. Use the actual sink to decide impact.

https://owasp.github.io/www-project-web-security-testing-guide/stable/4-Web_Application_Security_Testing/07-Input_Validation_Testing/17-Testing_for_Host_Header_Injection/

H44 - DNS concepts and facilities, RFC 1034

Consulted. DNS name space, resolvers and record concepts.

https://www.rfc-editor.org/rfc/rfc1034.html